# Assignment 6: Build and Evaluate Tree Models

Juan Maldonado Franco
DDS-8555 Predictive Analysis
Mohamed Nabeel

In [1]:
```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

ROOT = Path.cwd()
for parent in [ROOT, *ROOT.parents]:
    if (parent / "DDS-8555 - Predictive Analysis").exists():
        COURSE = parent / "DDS-8555 - Predictive Analysis"
        break
else:
    COURSE = ROOT.parents[1]
DATA = COURSE / "data"
KAGGLE = DATA / "kaggle"
SUBMISSIONS = DATA / "submissions"
RANDOM_STATE = 42
pd.set_option("display.max_columns", 80)
```

## Conceptual Question 1

A six-region recursive binary split can be created by first splitting X1 at t1, then splitting the left side on X2 at t2, and continuing with additional splits inside selected regions. The matching decision tree starts with the first X1 split at the root, then branches into the later X2 and X1 cuts. The important point is that each rectangular region corresponds to one terminal node, and each internal node corresponds to one binary decision.

## Applied Question 12 and Kaggle Tree Models

The applied exercise asks for boosting, bagging, random forests, and BART on a chosen data set. The Kaggle Obesity data is used here because it also satisfies the assignment competition requirement for decision tree, bagged tree, random forest, and boosted tree submissions.

In [2]:
```
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import BaggingClassifier, GradientBoostingClassifier, RandomF
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
```

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeClassifier

obesity = pd.read_csv(KAGGLE / "playground-series-s4e2" / "train.csv")
X = obesity.drop(columns=["NObeyesdad"])
y = obesity["NObeyesdad"]
cat = X.select_dtypes(include="object").columns.tolist()
num = [c for c in X.columns if c not in cat + ["id"]]
pre = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore"), cat)], rem
X_train, X_valid, y_train, y_valid = train_test_split(X.drop(columns=["id"]), y, te
base_tree = DecisionTreeClassifier(max_depth=8, random_state=RANDOM_STATE)
models = {
    "Decision tree": DecisionTreeClassifier(max_depth=8, random_state=RANDOM_STATE)
    "Bagging": BaggingClassifier(estimator=base_tree, n_estimators=80, random_state
    "Random forest": RandomForestClassifier(n_estimators=150, max_depth=12, random_
    "Gradient boosting": GradientBoostingClassifier(random_state=RANDOM_STATE),
}
rows = []
for name, clf in models.items():
    model = Pipeline([("pre", pre), ("model", clf)])
    model.fit(X_train, y_train)
    pred = model.predict(X_valid)
    rows.append({"model": name, "validation_accuracy": accuracy_score(y_valid, pred
pd.DataFrame(rows).sort_values("validation_accuracy", ascending=False)
```

Out[2]:

| | model | validation_accuracy |
|---|---|---|
| **3** | Gradient boosting | 0.905347 |
| **2** | Random forest | 0.890173 |
| **1** | Bagging | 0.888006 |
| **0** | Decision tree | 0.869461 |

In [3]:

```
from pathlib import Path
status_path = SUBMISSIONS / "kaggle_submission_status_playground-series-s4e2.txt"
text = status_path.read_text(encoding="utf-8", errors="ignore")
print("\n".join([line for line in text.splitlines() if "A6_" in line or "fileName"
```

# Interpretation

The tree family shows the bias-variance trade-off in a practical way. A single decision tree is interpretable but unstable. Bagging reduces variance by averaging many trees, random forests add feature randomness to reduce tree correlation, and boosting builds a sequence of trees that focus on difficult cases (Breiman, 1996, 2001; Friedman, 2001). The Kaggle evidence shows that all four required submissions completed, with boosted trees producing the best public score among this group.

# References

Breiman, L. (1996). Bagging predictors. *Machine Learning, 24*, 123-140. https://doi.org/10.1007/BF00058655

Breiman, L. (2001). Random forests. *Machine Learning, 45*, 5-32. https://doi.org/10.1023/A:1010933404324

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189-1232. https://doi.org/10.1214/aos/1013203451